Тема 4. Алгоритмы сортировки и поиска на массивах

Лекция 9

Алгоритмы поиска элемента в массиве

Целью данной **темы** является изучение алгоритмов сортировки и поиска элементов массива. В ней осуществляется постановка задачи сортировки, формулируются свойства алгоритмов сортировки, рассматриваются базовые алгоритмы: сортировка обменом, вставками и выбором, а так же их различные улучшения, позволяющие существенно сократить их временную сложность. Так же, в этой теме осуществляется постановка задачи поиска и рассматриваются алгоритмы линейного, блочного и бинарного поиска. Проводится сравнительный анализ всех рассмотренных алгоритмов.

4.5. Постановка задачи поиска элемента в массиве

Поиск необходимой информации в массиве данных одна из фундаментальных задач теоретического программирования. Приведем ее постановку.

Задача поиска элемента в массиве. Пусть есть массив a_0, a_1... a_{n-1}, состоящий из n элементов. Требуется установить, имеется ли в массиве элемент x, и, если имеется, то каков его индекс.

Как и в задаче сортировки, возможна ситуация, когда элементы состоят из нескольких полей, например, так, как показано на рис. 4.1. Таким образом, при обсуждении алгоритмов поиска мы предполагаем, что информация содержится в записях, составляющих некоторый список, который представляет собой массив данных в программе. Номера записей в массиве идут от 0 до n – 1, где n – полное число записей. В общем случае, поиск может осуществляться не во всем массиве, а только в некоторой

непрерывной его части, задаваемой нижнем и верхним индексами – *lb* (lower bound) и *ub* (upper bound) соответственно.

Из всех полей элемента массива нас будут интересовать значения лишь одного – ключевого поля. Обычно поиск производится не просто для проверки того, что нужный элемент в массиве имеется, но и для того, чтобы получить данные, относящиеся к этому значению ключа. Например, ключевое значение может быть номером сотрудника или порядковым номером, или любым другим уникальным идентификатором. После того, как нужный ключ найден, программа может, скажем, частично изменить связанные с ним данные или просто вывести всю запись. Во всяком случае, перед алгоритмом поиска стоит важная задача определения местонахождения ключа. Поэтому алгоритмы поиска возвращают индекс записи, содержащей нужный ключ. Если ключевое значение не найдено, то алгоритм поиска обычно возвращает значение индекса, превышающее верхнюю границу массива. Для наших целей мы будем предполагать, что элементы массива имеют номера от 0 до $n - 1$. Это позволит нам возвращать –1 в случае, если целевой элемент отсутствует в массиве.

Массив может быть не отсортированным или отсортированным по значению ключевого поля. В не отсортированном массиве порядок элементов случаен, а в отсортированном они идут в порядке возрастания ключа.

Отсортированный массив дает некоторое преимущество в организации алгоритма поиска, поскольку в этом случае имеется дополнительная информация о порядке следования элементов. Это позволяет создавать более эффективные алгоритмы поиска. Далее мы рассмотрим три алгоритма поиска: *линейный, блочный* и *бинарный*. Первый из них не имеет ограничения на упорядоченность массива, а два следующих могут применяться только в отсортированных массивах.

4.6. Алгоритмы линейного, блочного и бинарного поиска

Линейный поиск. Работа алгоритма заключается в том, что элементы массива, начиная с первого *a[lb]*, последовательно сравнивается с искомым элементом. Сравнение элементов продолжается до тех пор, пока не будут просмотрены все элементы *a[lb]…a[ub]*, или очередной элемент массива не

равен искомому. По завершении цикла поиска анализируется ситуация, в которой он был завершен. Если был найден целевой элемент, то решением является его индекс, а, если был достигнут конец последовательности, то решением является –1.

В алгоритме линейного поиска выполняется проверка на достижение конца просматриваемой последовательности. Эта проверка осуществляется каждый раз перед обращением к очередному элементу. Можно слегка улучшить этот алгоритм, устранив проверки на достижение конца последовательности.

Проверим последний элемент последовательности. Если он совпадает с искомым, то решением будет индекс *ub*. Иначе, заменим элемент *a[ub]* на искомый. Тогда проверка на окончание последовательности становится не нужной. Гарантированно целевой элемент будет найден. Если этот элемент находится внутри последовательности, то результатом будет его индекс. Если же этим элементом оказался *a[ub]*, то искомого элемента в массиве нет и решение есть –1.

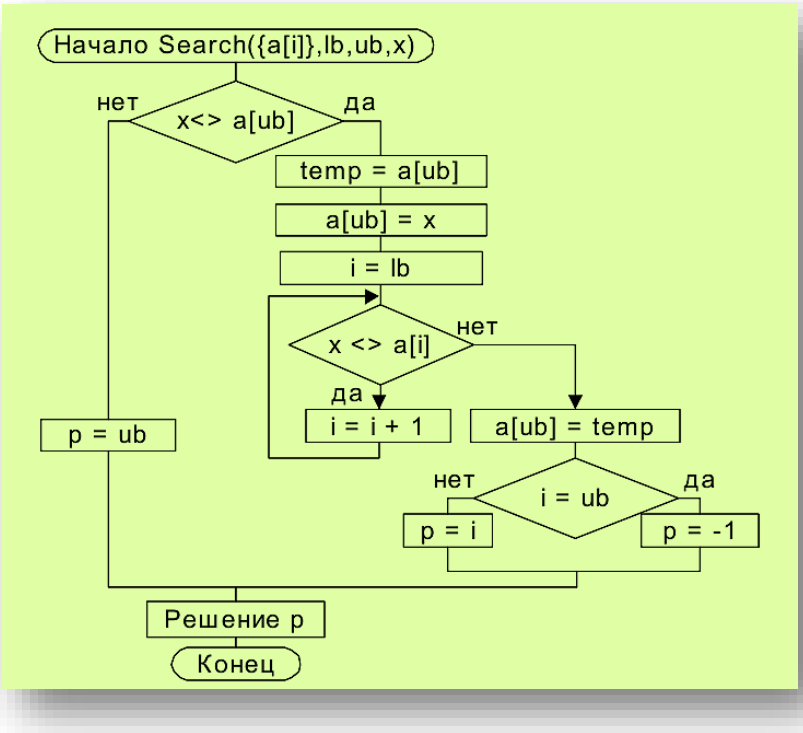


Рис. 4.27. Блок-схема алгоритма линейного поиска

Естественно, что перед завершением алгоритма необходимо вернуть на старое место прежний элемент *a[ub]*. На рис. 4.27 приведена блок-схема алгоритма линейного поиска, учитывающая сделанное улучшение.

Блочный поиск. Идея алгоритма блочного поиска состоит в том, что массив, упорядоченный по возрастанию, разбивается на определенное число блоков. В процессе поиска целевой элемент последовательно сравнивается с последним элементом блоков. Если целевой элемент меньше последнего элемента очередного блока, то искомый элемент может находиться только внутри этого блока. Для поиска элемента в блоке можно применить линейный поиск.

При реализации алгоритма блочного поиска ставится вопрос о количестве и размере выделяемых блоков. Желательно, чтобы все блоки имели одинаковый размер. Пусть последовательность состоит из *n* элементов и при поиске разбивается на *m* равных по размеру блоков (может быть за исключением последнего). Тогда в каждом блоке будет не более *n/m* элементов. В худшем случае, если искомый элемент окажется в последнем блоке, то для поиска блока потребуется просмотреть *m* элементов массива. Выполнить линейный поиск в блоке можно, просмотрев не более чем *n/m* элементов. Следовательно, общее число обрабатываемых элементов не превышает *m* + *n/m*. Эта величина зависит от числа блоков и будет минимальна, когда ее производная равна нулю, т.е. при $m = \sqrt{n}$. Учитывая, что количество боков должно быть целым числом, в качестве значения *m* следует выбрать $\lfloor \sqrt{n} \rfloor$ – наибольшее целое, не превышающее $\sqrt{n}$. Это значение представляет собой размер блока. Когда $\lfloor \sqrt{n} \rfloor = \sqrt{n}$ количество блоков в точности совпадает с *m*. В случае когда $\lfloor \sqrt{n} \rfloor \prec \sqrt{n}$, количество боков больше *m*, но не больше чем на два.

На рис. 4.28 приведена бок-схема алгоритма блочного поиска. В этом алгоритме рассчитывается размер блока $h = \sqrt{n}$. И далее в цикле, пока не будет найден блок, потенциально содержащий целевой элемент, или индекс последнего элемента текущего блока не выйдет за границы последовательности, просматривается последний элемент очередного блока.

Если этот элемент больше или равен целевому, то к текущему блоку применяется алгоритм линейного поиска. Если среди рассмотренных блоков не нашлось блока, потенциально содержащего целевой элемент и имеется еще не просмотренный последний блок последовательности, то он анализируется аналогичным образом. В случае, когда целевой элемент оказывается больше *a[ub]* решение есть –1.

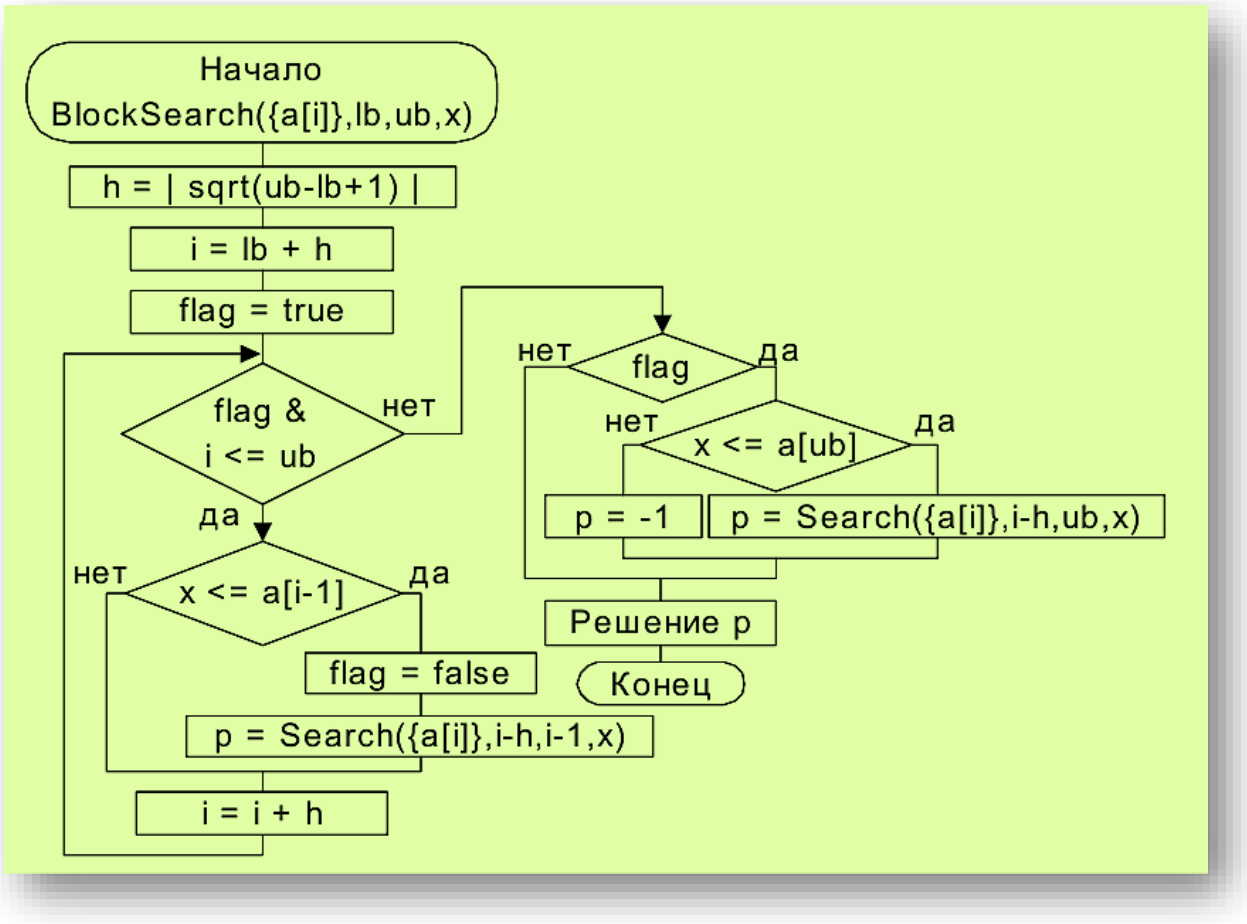


Рис. 4.28. Блок-схема алгоритма блочного поиска

Бинарный поиск подобно блочному поиску может использоваться только для упорядоченного массива. Идея этого алгоритма заключается в следующем.

Имеем массив *{a[i]}* из *n* элементов. Пусть индексы в концах анализируемой части массива: *lb*=0, *ub*=*n*-1. Вычислим индекс серединного элемента анализируемой части массива *m=lb+(ub-lb) div 2*. Сравним значение в серединном элементе с искомым значением *x*. Если совпадение найдено, решением является *m*. Если *a[m]* < *x*, то следует проводить повторный поиск в правой половине рассматриваемого массива. Если *a[m]>x*, то проводить повторный поиск в левой половине рассматриваемого массива. Если искомый элемент не находится в массиве, то возвратить индикатор сбоя.

Рассмотрим пример бинарного поиска. Дан массив целых чисел A. Найти элемент с заданным значением 33. На рис. 4.29 представлены этапы бинарного поиска этого элемента.

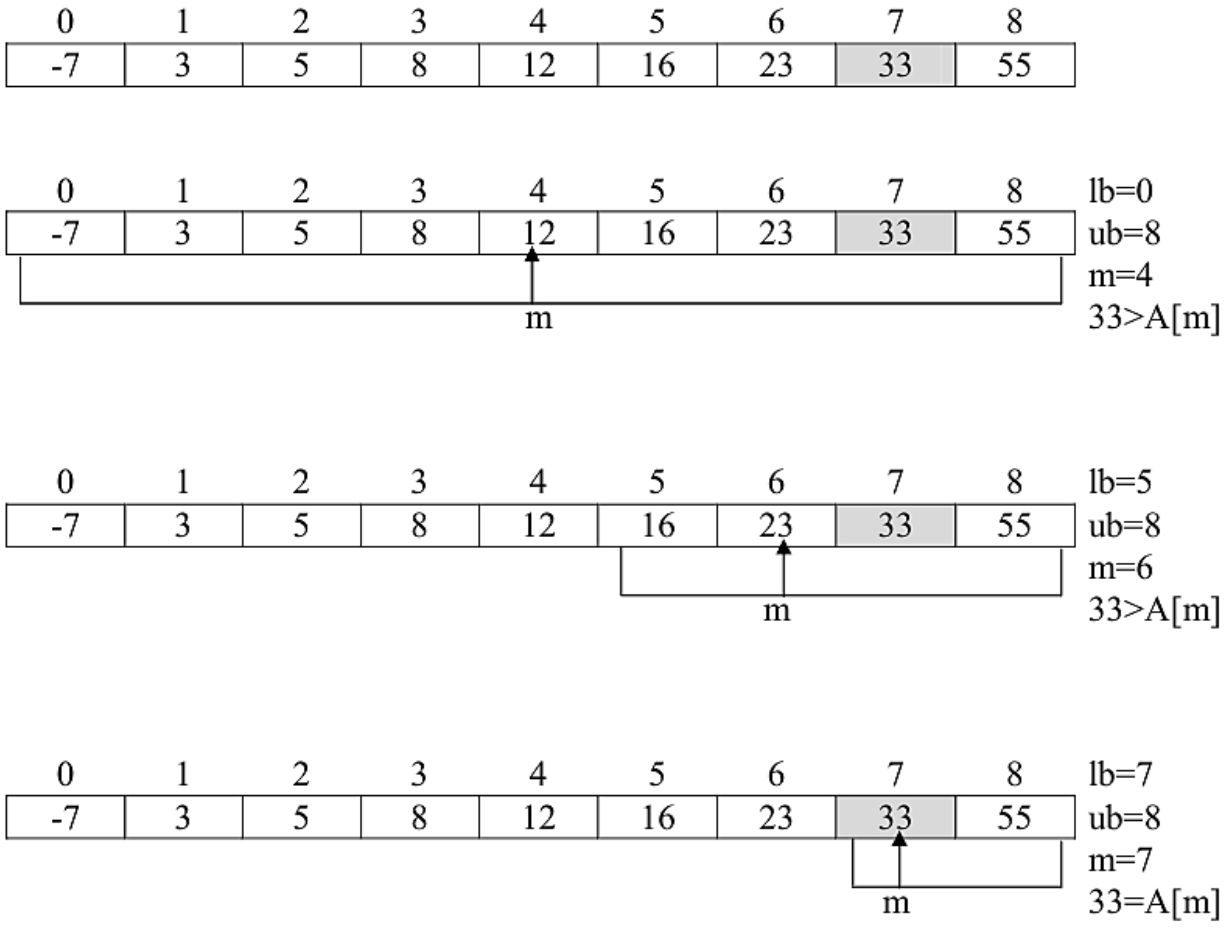


Рис. 4.29. Пример бинарного поиска элемента в массиве

При последовательном поиске в этом случае требуется 8 сравнений, а при бинарном поиске – только три.

На рис. 4.30 приведена блок-схема алгоритма бинарного поиска.

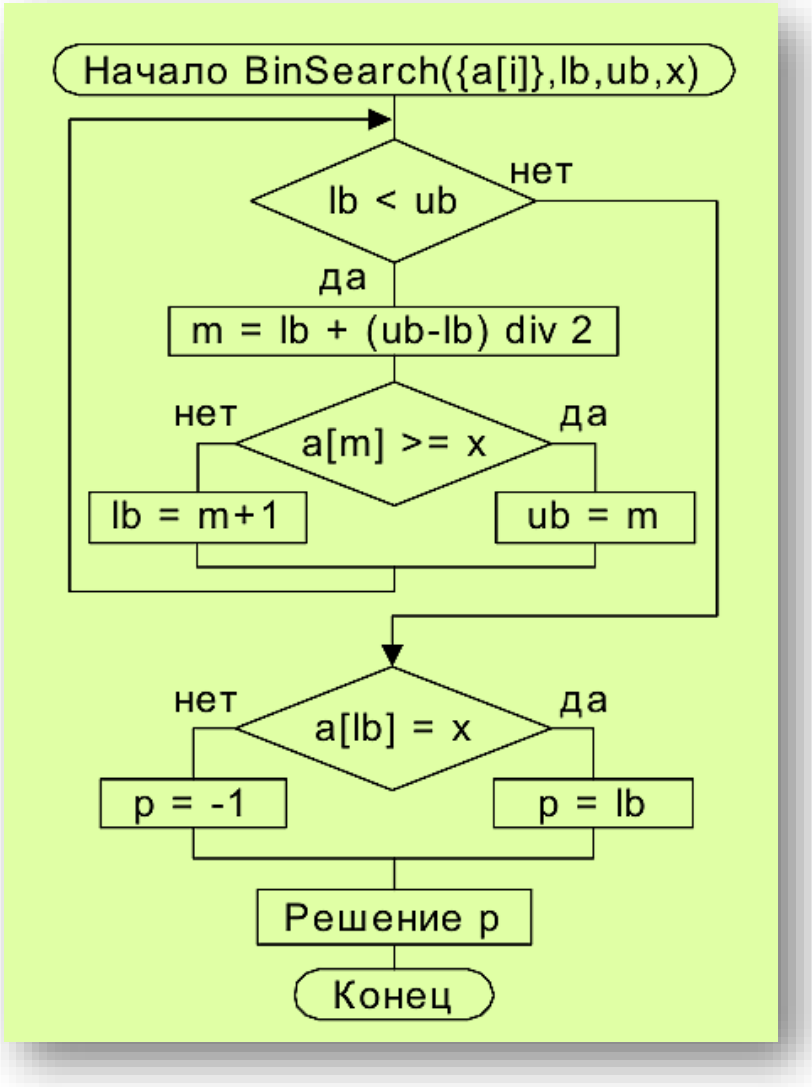


Рис. 4.30. Блок-схема алгоритма бинарного поиска

Вопросы для повторения темы 4

1. В чем заключается задача сортировки?
2. Какими параметрами обладают алгоритмы сортировки?
3. В чем заключается алгоритм сортировки обменом?
4. В чем заключается алгоритм сортировки вставками?
5. В чем заключается алгоритм сортировки выбором?
6. Какие модификации базового алгоритма сортировки обменом вы знаете? В чем их преимущество?
7. В чем заключается быстрая сортировка? Какова ее временная сложность?
8. Какие модификации базового алгоритма сортировки вставками вы знаете? В чем их преимущество?
9. В чем заключается сортировка Шелла? Какова ее временная сложность?
10. Что такое пирамида? Как эта структура применяется для сортировки?
11. В чем заключается пирамидальная сортировка? Какова ее временная сложность?
12. В чем заключается задача поиска элемента в массиве?
13. Какие алгоритмы поиска вы знаете? Какой из них является наиболее быстродействующим?